Tutorial Questions | Week 10

COSC2779 - Deep Learning

This tutorial reviews deep generative models: autoencoders and variational autoencoders, generative adversarial networks, and diffusion models. Please try the questions before you join the session. The question below has a similar structure to the case study questions that you can expect in the end semester test.

1. You have been hired by a hospital research group that wants to build a **generative model of chest X-ray images**. They have a large set of unlabelled images. They have several goals in mind: (i) detecting unusual (anomalous) scans, (ii) **image retrieval** – given a query scan, find the most visually similar scans in the archive – and later (iii) generating realistic synthetic scans to augment a small labelled dataset for a downstream classifier. You will work through the main families of generative models to decide what fits.

   (a) You first train a plain **autoencoder** (encoder → bottleneck → decoder) on the images with a reconstruction loss. Explain how the same trained model can be used for goals (i) anomaly detection and (ii) image retrieval, and name a suitable reconstruction loss.

   > **Solution:**
   >
   > - **Anomaly detection**: the model reconstructs normal scans well but reconstructs unusual ones poorly, so a high **reconstruction error** flags an anomaly (threshold it).
   > - **Image retrieval**: encode every scan to its **bottleneck code** z (a compact feature vector) once and store it; for a query scan, encode it and return the archive scans whose codes are **nearest** (e.g. by Euclidean or cosine distance). Similar images map to nearby codes, so nearest neighbours in latent space are visually similar scans.
   > - **Loss**: pixel reconstruction error, e.g. **MSE** (or binary cross-entropy for images scaled to $[0, 1]$).

   (b) You now try to generate new scans by sampling a random point in the autoencoder's bottleneck and decoding it – but you get unrealistic junk. Why does a plain autoencoder fail as a generator?

   > **Solution:** The autoencoder is only trained to reconstruct; nothing constrains the **structure of the latent space**. Encoded points sit in arbitrary regions with **gaps** and no consistent scale, so a random z usually lands in "empty" space the decoder never saw. There is no distribution to sample from.

   (c) A **variational autoencoder (VAE)** fixes this. What two changes does it make, and what is the role of each term in its loss $\mathcal{L} = \underbrace{\mathbb{E}[-\log p(x|z)]}_{\text{reconstruction}} + \underbrace{D_{KL}(q(z|x) \,\|\, \mathcal{N}(0, I))}_{\text{KL}}$?

   > **Solution:**
   >
   > - Encode each input as a **distribution** $\mathcal{N}(\mu, \sigma^2)$ rather than a point, and add a **KL term** pulling those posteriors towards a standard normal prior.
   > - **Reconstruction term**: keeps decoded samples faithful to the input.
   > - **KL term**: regularises the latent so it is smooth and gap-free and matches $\mathcal{N}(0, I)$ – so we can **sample** $z \sim \mathcal{N}(0, I)$ and decode to get new scans.

   (d) Training a VAE needs to back-propagate through a sampling step $z \sim \mathcal{N}(\mu, \sigma^2)$. What trick makes this possible, and why is it needed?

   > **Solution:** The **reparameterisation trick**: write $z = \mu + \sigma \odot \epsilon$ with $\epsilon \sim \mathcal{N}(0, I)$. Sampling randomness is moved into ϵ, so z is a differentiable function of μ, σ and gradients can flow to the encoder. (You cannot back-propagate through a raw random sampling operation.)

(e) VAE samples come out realistic but **blurry**. A colleague suggests a **GAN** instead. Describe the two networks and the objective they optimise.

Solution: A **generator** G maps noise z to images; a **discriminator** D tries to tell real images from generated ones. They play a **minimax** game:

$$\min_G \max_D \; \mathbb{E}_x[\log D(x)] + \mathbb{E}_z[\log(1 - D(G(z)))].$$

D learns to classify real vs fake; G learns to produce images that fool D. Adversarial training pushes G toward sharp, realistic samples (no pixel-averaging blur).

(f) During GAN training you notice the generator only ever produces a couple of very similar scans. What is this failure called, and give one reason it happens.

Solution: Mode collapse: G maps many inputs to a few outputs, covering only part of the data distribution. It happens because those few samples are enough to **fool the current** D, so G has no incentive to be diverse (the adversarial game gets stuck). GAN training is also generally **unstable**/hard to balance.

(g) You move to a **diffusion model**. Describe its **forward** process and its **reverse** process, and say which one is learned.

Solution:

- **Forward** (fixed, not learned): gradually add Gaussian noise to an image over T steps until it is (almost) pure noise – a simple, parameter-free schedule.
- **Reverse** (**learned**): a network removes a little noise at each step, so starting from noise and denoising repeatedly produces a new sample.

(h) In the DDPM formulation, what does the network actually predict, and what loss is used? Why is this attractive compared to a GAN?

Solution: The network $\epsilon_\theta(x_t, t)$ predicts the **noise** that was added at step t; training minimises a simple regression loss

$$\mathcal{L} = \mathbb{E}_{x_0,t,\epsilon}\big[\,\|\epsilon - \epsilon_\theta(x_t, t)\|^2\,\big], \qquad x_t = \sqrt{\bar{\alpha}_t}\,x_0 + \sqrt{1-\bar{\alpha}_t}\,\epsilon.$$

It is a **stable** MSE objective (no adversarial game, no posterior to approximate), and tends to give **high quality and excellent diversity** – avoiding mode collapse.

(i) What kind of network is typically used for ϵ_θ on images, and why is that architecture a good fit? What is the main practical drawback of diffusion at generation time?

Solution: A **U-Net** (encoder–decoder with **skip connections**): the input and output are both images of the same size, and skip connections preserve fine spatial detail while the bottleneck captures context; the timestep t is fed in as an embedding.

Drawback: **slow sampling** – generating one image needs many sequential network evaluations (large T). (Faster samplers like DDIM reduce this.)

(j) For goal (iii) the group needs to generate scans of a specific finding (e.g. "pneumonia"). How would you make any of these generators produce a chosen class rather than a random sample?

Solution: Make it **conditional**: feed the class label (or a text embedding) y into the generator.

- **Conditional GAN**: give y to both G and D.

- **Conditional VAE / diffusion**: condition the decoder / the denoiser $\epsilon_\theta(x_t, t, y)$ on y; diffusion often adds **classifier-free guidance** to strengthen adherence to y.

(k) Summarise the trade-offs: rank VAE, GAN, and diffusion on (a) training stability, (b) sample quality/diversity, and (c) sampling speed. Which would you recommend here, and why?

Solution:

- **Stability**: VAE and diffusion stable; GAN unstable.
- **Quality/diversity**: diffusion best (sharp + diverse); GAN sharp but risk of mode collapse; VAE diverse but blurry.
- **Sampling speed**: VAE and GAN fast (one pass); diffusion slow (many steps).

Recommendation: for high-quality, diverse synthetic scans to augment training data, **diffusion** is the strongest choice despite slow sampling (a one-off cost). If fast sampling or a usable latent space matters more, a GAN or VAE respectively may be preferred.